

VULNERABILITY ASSESSMENT REPORT

Insecure Direct Object Reference (IDOR) in Basket Retrieval

OWASP Juice Shop

Find it. Prove it safely. Explain it like a professional.

Case File:	CA-WEB-03
Intern ID:	CA0145
Target application:	OWASP Juice Shop (local instance)
Tested endpoint:	GET /rest/basket/{id}
Deliverables:	Vulnerability Assessment Report Proof of Concept Risk Rating Remediation Recommendations

PREPARED BY: **Nataliah Alcala-Maharaj**

Web App Security — Vulnerability Assessment & Professional Reporting, Project 03

Testing was performed only against a locally hosted, intentionally vulnerable application. No external or production systems were assessed.

Table of Contents

1. Document Control.....3

2. Executive Summary3

3. Finding Summary.....3

4. Scope and Methodology..... 3 - 4

5. Detailed Finding - WEB-01 5 - 6

6. Proof of Concept6

7. Evidence 7 - 9

8. Remediation Recommendations 10 - 11

9. Conclusion..... 11

10. References 11

1. Document Control

Report title	Insecure Direct Object Reference (IDOR) in Basket Retrieval
Assessment target	OWASP Juice Shop - local instance at http://127.0.0.1:3000
Assessment type	Authorized academic web application security exercise
Assessment date	1 August 2026
Prepared by	Nataliah Alcala-Maharaj
Tools used	Burp Suite Community Edition v2025.7.4, Google Chrome, Kali Linux, Oracle VirtualBox
Report version	1.0
Data handling	Authentication tokens and session cookies have been redacted from all evidence

2. Executive Summary

The assessment of the locally hosted OWASP Juice Shop application identified an insecure direct object reference (IDOR) in the basket retrieval function. The application used the numeric basket ID provided in the URL to retrieve a basket but did not verify that the basket belonged to the authenticated user.

During testing, Test User B was logged in and assigned to basket 8. The request was sent to Burp Suite Repeater, where the basket ID was changed from 8 to 7 without changing Test User B’s authentication details. The application returned an HTTP 200 OK response and displayed basket 7, which belonged to Test User A. This shows that one authenticated user could access another user’s basket because the application did not perform the required object-level authorization check.

Overall risk rating: MEDIUM

The demonstrated impact was unauthorized read access to another user's basket data. Modification, deletion, checkout manipulation, and other write operations were not tested and are not claimed in this report.

3. Finding Summary

ID	Finding	Severity	Affected endpoint	Status
WEB-01	IDOR permits access to another user's basket	Medium	GET /rest/basket/{id}	Confirmed

4. Scope and Methodology

4.1 Scope

- In scope: basket retrieval functionality on the locally hosted OWASP Juice Shop instance at 127.0.0.1:3000.
- Test accounts: two researcher-controlled dummy user accounts, identified in this report as Test User A and Test User B.

- Out of scope: external systems, production systems, denial-of-service activity, destructive testing, and unauthorized access to real user data.

4.2 Assessment Methodology

The assessment used a controlled two-account authorization test. Each test account was given a distinct basket and product so that ownership could be verified. Requests were captured through Burp Suite, replayed in Repeater, and modified one variable at a time. Authentication values were deliberately kept unchanged during the cross-account test.

- Test User A: basket 7, internal UserId 7, containing “Fruit Press”.
- Test User B: basket 8, internal UserId 8, containing “Basil Smoothie”.
- Manipulated value: the basket identifier in the URL path only.

4.1 Figure 1 - Test User A's basket

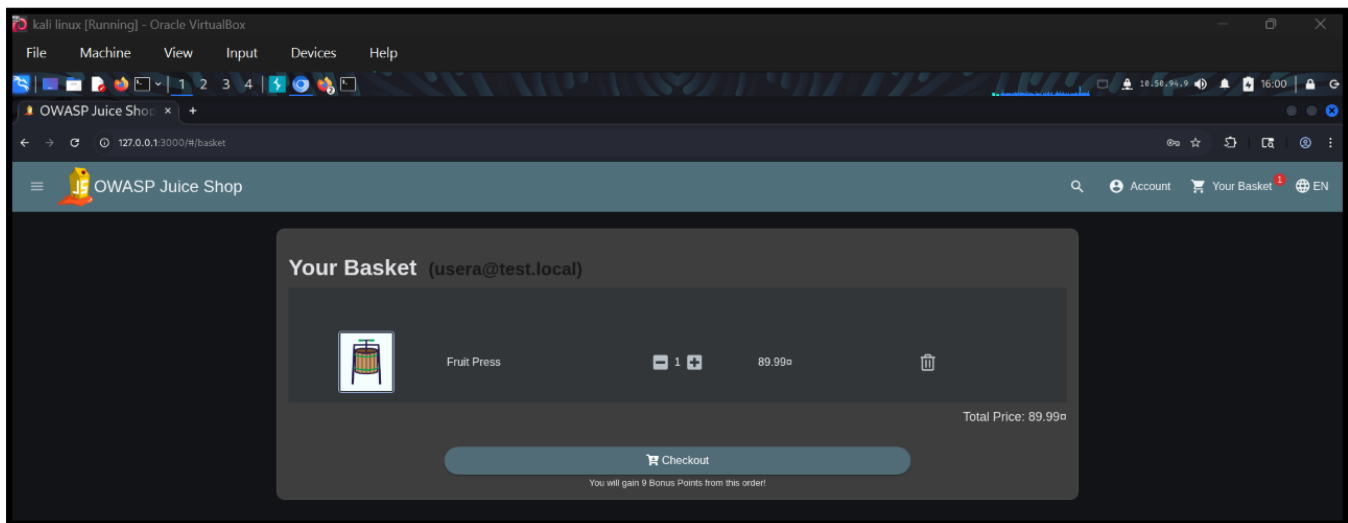


Figure 1: Test User A's basket (ID 7) as shown in the application UI, confirming the "Fruit Press" contents described above.

4.2 Figure 1 - Test User B's basket

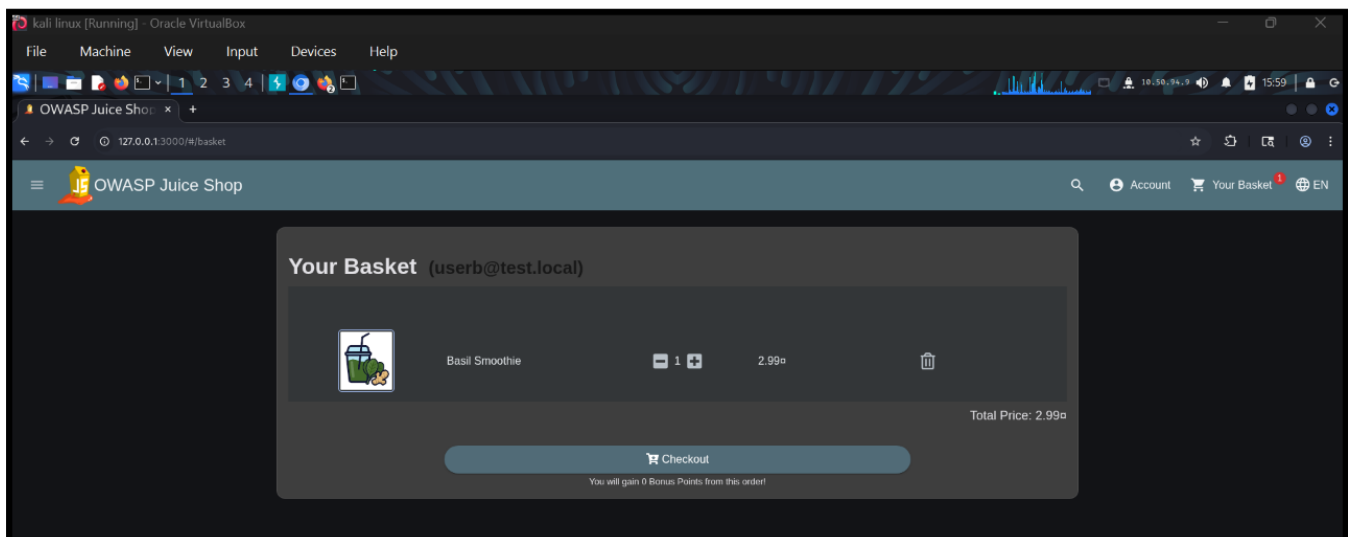


Figure 2: Test User B's basket (ID 8) as shown in the application UI, confirming the "Basil Smoothie" contents described above.

5. Detailed Finding - WEB-01

Finding	Insecure Direct Object Reference in Basket Retrieval
Severity	Medium
Provisional CVSS v3.1	4.3 - AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Affected endpoint	GET /rest/basket/{basketId}
Authorization type	Horizontal privilege escalation
CWE	CWE-639 - Authorization Bypass Through User-Controlled Key
OWASP mapping	A01:2025 Broken Access Control; API1:2023 Broken Object Level Authorization
Authentication required	Yes - tested with an authenticated standard user

5.1 Vulnerability Description

The basket retrieval endpoint uses a client-supplied numeric identifier to select a basket. Although the request includes a valid authentication token, the server does not verify that the requested basket is owned by the authenticated account. As a result, an authenticated user can alter the identifier and retrieve another user's basket.

The vulnerability is not caused merely by the use of sequential identifiers. Predictable IDs make discovery easier, but the underlying security failure is the absence of a server-side object ownership check on every request.

5.2 Why The Vulnerability Occurs

- The application trusts the basket ID supplied in the URL path.
- Authentication confirms who the requester is, but the endpoint does not enforce authorization for the specific basket object.
- The requested object is returned based on its identifier without binding the database lookup to the authenticated user's identity.

5.3 Security Impact

A malicious authenticated user could enumerate basket identifiers and access basket information belonging to other users. The confirmed exposure includes basket contents, product details, basket identifiers, and internal user identifiers. This represents a confidentiality breach and horizontal privilege escalation.

In this assessment, the exposed information was limited to product names, prices, and quantities. However, the main problem is the missing ownership check at the endpoint, not the type of data stored in the basket. In a production e-commerce application, the same insecure access-control pattern could affect other endpoints and expose more sensitive information, such as order history, saved addresses, or payment-related data.

Read-level IDOR exposure also carries risk beyond the directly disclosed data. An attacker able to enumerate basket identifiers can harvest information about other users at scale, which can support follow-up activity such as targeted social engineering or further reconnaissance against the application, even where no data is directly modified.

Scope limitation: This assessment confirmed unauthorized read access only. It did not test whether the same authorization weakness affects basket modification, deletion, quantity changes, checkout, or other HTTP methods. Those impacts require separate validation.

5.4 Affected Systems And Users

- The GET /rest/basket/{basketId} endpoint in the tested Juice Shop instance.
- Authenticated users whose basket identifiers can be predicted or discovered.
- Any application component or API operation that reuses the same basket lookup logic without an ownership check.

6. Proof of Concept

6.1 Preconditions

- Two researcher-controlled Juice Shop accounts were created.
- Each account was assigned a distinct basket and product.
- Burp Suite proxying was enabled, and requests were captured from the local application.

6.2 Steps to Reproduce

1. Confirm Test User A legitimately owns basket 7 (UserId 7) containing “Fruit Press”.
2. Confirm Test User B legitimately owns basket 8 (UserId 8) containing “Basil Smoothie”.
3. Capture Test User B's GET /rest/basket/8 request and send it to Burp Repeater.
4. Change only the path to /rest/basket/7, leaving Test User B's Authorization header and session cookies unchanged, then send the request.
5. Observe HTTP 200 OK with basket 7, UserId 7, and Test User A's “Fruit Press” product.

6.3 Expected And Actual Results

Expected secure result	Actual result
The server verifies that basket 7 belongs to the authenticated user. Because Test User B does not own basket 7, the application returns 403 Forbidden or a non-enumerating 404 Not Found response.	The server returns HTTP 200 OK and discloses basket 7 to Test User B.
The response does not include data belonging to another account.	The response includes internal UserId 7 and the “Fruit Press” product associated with Test User A.

6.4 Sanitized Request And Response Excerpts

```
LEGITIMATE USER B
GET /rest/basket/8 HTTP/1.1
Authorization/Cookie: [REDACTED]
HTTP/1.1 200 OK
{ "data": { "id": 8, "UserId": 8, ... } }

TAMPERED WHILE STILL AUTHENTICATED AS USER B
GET /rest/basket/7 HTTP/1.1
Authorization/Cookie: [SAME USER B VALUES - REDACTED]
HTTP/1.1 200 OK
{ "data": { "id": 7, "UserId": 7, "Products": [{ "name": "Fruit Press" }] } }
```

7. Evidence

All screenshots below were captured from the authorized local lab. Authentication tokens and session cookies were redacted before inclusion.

7.1 Figure 3 - Legitimate Basket Access (Test User A)

The screenshot displays the Burp Suite interface with a list of HTTP history items. The selected item is a GET request to `/rest/basket/7` with a status code of 200. The response is a 200 OK with a JSON body. The response headers include `Access-Control-Allow-Origin: *`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Feature-Policy: payment 'self'`, `X-Recruiting: /#/jobs`, `Content-Type: application/json; charset=utf-8`, `Content-Length: 583`, `ETag: W/"247-0v1V9wohPIceR50wOtjB/nR7U0"`, `Vary: Accept-Encoding`, `Connection: keep-alive`, and `Keep-Alive: timeout=5`. The JSON body contains the following data:

```
{
  "status": "success",
  "data": {
    "id": 7,
    "coupon": null,
    "userId": 25,
    "createdAt": "2026-08-01T19:56:15.683Z",
    "updatedAt": "2026-08-01T19:56:15.683Z",
    "products": [
      {
        "id": 25,
        "name": "Fruit Press",
        "description": "Fruits go in. Juice comes out. Pomace you can send back to us for recycling purposes.",
        "price": 89.99,
        "deluxePrice": 99.99,
        "image": "fruit_press.jpg",
        "createdAt": "2026-08-01T19:52:51.099Z",
        "updatedAt": "2026-08-01T19:52:51.099Z",
        "deletedAt": null
      }
    ]
  }
}
```

Figure 3. Test User A legitimately retrieves basket 7, which returns internal `UserId` 7 and the “Fruit Press” product.

7.2 Figure 4 - Legitimate Basket Access (Test User B)

The screenshot displays the Burp Suite interface with the HTTP history tab selected. A table of intercepted requests is shown, with the request to `/rest/basket/8` highlighted. The details pane shows the request and response for this specific request.

Request Details:

- Method: GET
- URL: `/rest/basket/8`
- Host: `127.0.0.1:3000`
- Headers: `sec-ch-ua-platform: "Linux"`

Response Details:

- Status: 200 OK
- Content-Type: `application/json; charset=utf-8`
- Content-Length: 561
- ETag: `W/"251-orRLBhSPV19hKXUAgW5zzEgI7o"`
- Vary: `Accept-Encoding`
- Date: `Sat, 01 Aug 2026 19:58:50 GMT`
- Connection: `keep-alive`
- Keep-Alive: `timeout=5`

Response Body (JSON):

```
{
  "status": "success",
  "data": {
    "id": 8,
    "coupon": null,
    "userId": 26,
    "createdAt": "2026-08-01T19:57:21.628Z",
    "updatedAt": "2026-08-01T19:57:21.628Z",
    "products": [
      {
        "id": 52,
        "name": "Basil Smoothie",
        "description": "A unique blend of fresh basil and ginger for a healthy kick.",
        "price": 2.99,
        "deluxePrice": 2.99,
        "image": "basil_smoothie.png",
        "createdAt": "2026-08-01T19:52:51.104Z",
        "updatedAt": "2026-08-01T19:52:51.104Z",
        "deletedAt": null,
        "BasketItem": {}
      }
    ]
  }
}
```

Figure 4. Test User B legitimately retrieves basket 8, which returns internal UserId 8 and the “Basil Smoothie” product.

7.3 Figure 5 - Exploit Request in Burp Repeater

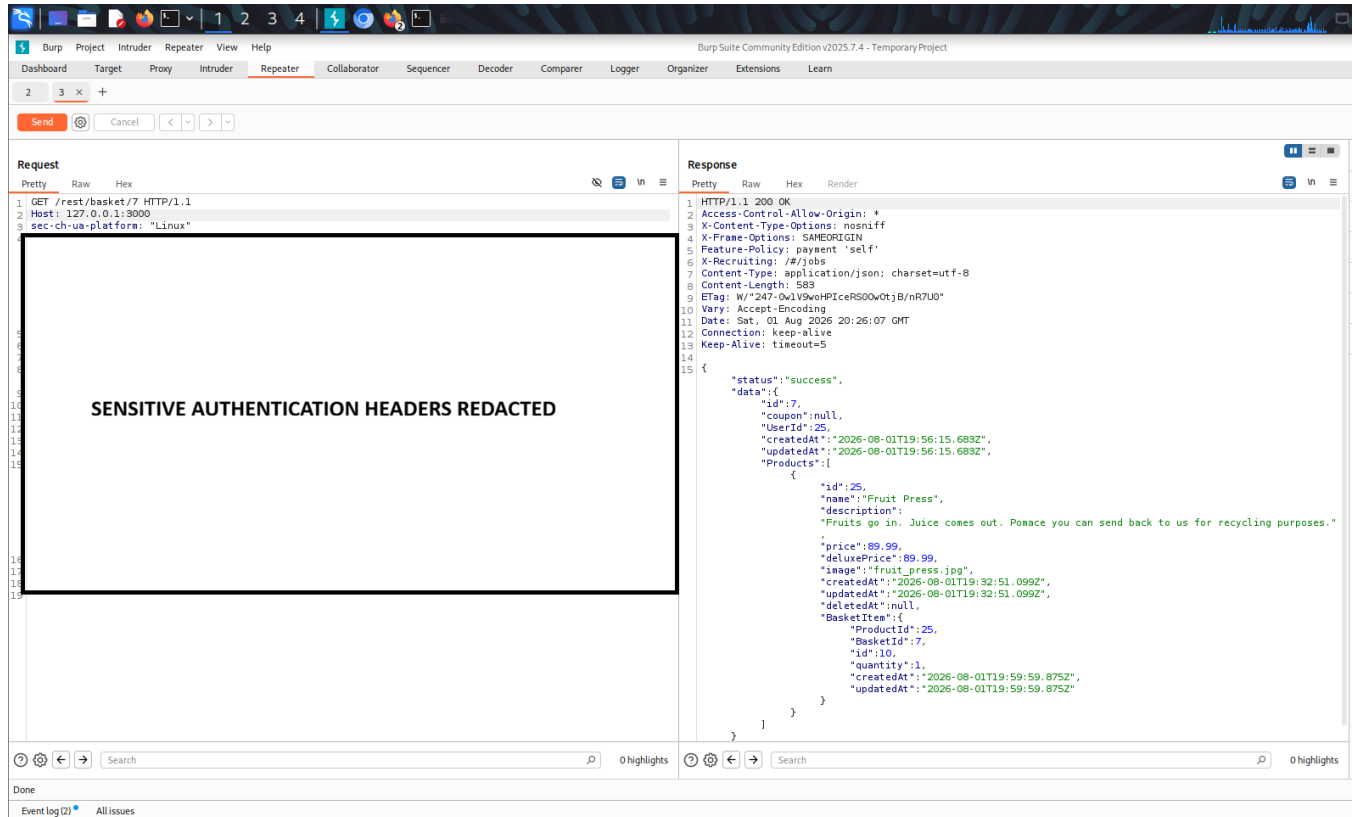


Figure 5. In Burp Repeater, the request is changed to `GET /rest/basket/7` while Test User B remains authenticated. The server returns HTTP 200 OK and discloses Test User A's basket data.

8. Remediation Recommendations

8.1 Enforce Object-Level Authorization

For every request that accesses a basket, the server must verify that the basket belongs to the authenticated user before returning or modifying it. Authentication alone is not sufficient; authorization must be evaluated for the specific object on every request.

Conceptual secure lookup

```
requested_basket = find basket where:  
  basket.id == supplied_basket_id  
  AND basket.UserId == authenticated_user.id  
  
if no matching basket exists:  
  return 404 Not Found # or 403 Forbidden according to application policy  
  
return requested_basket
```

8.2 Prefer Server-Derived Basket Selection

Where the business workflow allows it, avoid accepting an arbitrary basket identifier from the client. Derive the current basket from the authenticated account or session so that the client cannot select another user's basket.

8.3 Apply Authorization Consistently

- Use a centralized authorization service, middleware, or policy layer rather than duplicating ownership checks inconsistently across controllers.
- Apply the same ownership validation to read and write operations, including GET, POST, PUT, PATCH, and DELETE requests.
- Adopt deny-by-default behavior when the ownership relationship cannot be established.

8.4 Add Automated Authorization Tests

- Create integration tests with at least two users and verify that User A cannot retrieve or modify User B's basket, and vice versa.
- Test valid IDs, invalid IDs, sequential IDs, stale IDs, and IDs belonging to deleted or inactive accounts.
- Include negative authorization tests in continuous integration so future code changes cannot reintroduce the flaw.

8.5 Use Opaque Identifiers Only As Defense In Depth

Replacing sequential IDs with random or opaque identifiers can make enumeration more difficult, but it does not correct the authorization failure. Ownership checks remain mandatory even when UUIDs or other non-sequential values are used.

8.6 Monitoring And Detection

- Log authorization failures and repeated attempts to request multiple basket IDs from the same account or session.
- Alert on abnormal sequential object access patterns where appropriate.
- Do not log raw authentication tokens or sensitive session values.

8.7 Retest Criteria

The finding can be considered remediated when an authenticated user is only able to access their own basket, even if they change the basket ID in the request. Retesting should also confirm that actions such as adding, updating, or removing basket items apply the same ownership checks and that normal basket functions continue to work correctly.

9. Conclusion

The assessment confirmed a horizontal access-control vulnerability in the Juice Shop basket retrieval endpoint. By changing a user-controlled basket identifier while preserving the authentication context of another account, the application returned data belonging to a different user. The result demonstrates that the server authenticated the requester but failed to authorize access to the requested basket object.

The primary corrective action is to enforce server-side ownership checks on every basket operation and, where possible, derive the active basket from the authenticated user rather than from an arbitrary client-supplied identifier. Implementing negative authorization tests will help prevent recurrence.

10. References

1. OWASP. "Insecure Direct Object Reference Prevention Cheat Sheet."
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html
2. OWASP. "A01:2025 - Broken Access Control." https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
3. OWASP API Security Project. "API1:2023 - Broken Object Level Authorization." <https://owasp.org/API-Security/editions/2023/en/Oxa1-broken-object-level-authorization/>
4. MITRE. "CWE-639: Authorization Bypass Through User-Controlled Key."
<https://cwe.mitre.org/data/definitions/639.html>